Reinforcement Learning with Test Rewards for x86-to-ARM Assembly Transpilation

Mid-point Report

Enzo Shiraishi Nunes da Silva
Mohammed Bin Zayed University of Artificial Intelligence
enzo.shiraishi@mbzuai.ac.ae
PhD in Computer Science

Sergei Pasynkov
Mohammed Bin Zayed University of Artificial Intelligence
sergei.pasynkov@mbzuai.ac.ae
PhD in Computer Science

Zayed Khaled Al Tamimi
Mohammed Bin Zayed University of Artificial Intelligence
zayed.altamimi@mbzuai.ac.ae
PhD in Computer Science

1 Introduction

Many x86 programs are still in use without their source code, so they cannot simply be recompiled for ARM processors. Dynamic binary translators such as Apple's Rosetta 2 run them anyway, at a cost paid on every execution. A one-time static translation avoids this cost. Guaranteed Guess (GG) uses a small language model to translate x86 assembly into ARM assembly, and accepts a translation only if it passes the unit tests of the program [10]. On small benchmark programs, the authors report that the translated code runs 1.73 times faster than the x86 original under Rosetta 2.

GG is trained with supervised fine-tuning (SFT) on pairs of x86 and ARM assembly produced by a compiler, so its loss rewards reproducing the compiler's ARM output token by token. Correctness, however, depends only on behavior, and many different instruction sequences behave identically. The GG authors show correct translations that use other registers, other stack offsets, or a shift and an addition instead of a multiplication, and the SFT loss penalizes each of them. Conversely, a single wrong immediate value changes the behavior of a program but is penalized about as much as a harmless register rename. SFT also trains the model only on reference prefixes, while at generation time it continues from its own output. The weakness is clearest at optimization level O2, where the released model passes 45% of the HumanEval-C problems, compared with 99% at O0 [10]. The unit tests, in contrast, measure behavior directly. They run the translated code on several inputs and execute over 97% of its lines [10]. In GG, however, they are used only to accept or reject a translation after it is generated.

We use the tests as a training signal. Starting from the released GG checkpoint for O2, we train the model with Group Relative Policy Optimization (GRPO) [23], where a translation receives a reward of 1 if it passes all tests and 0 otherwise. Any correct translation gets the full reward, and the model learns from its own outputs. GRPO also needs some attempts at a problem to pass and others to fail, which happens at O2 but rarely at O0. Our main research question is whether this training makes the model solve more problems than the released SFT checkpoint. Reinforcement learning with binary rewards mainly raises the probability of solutions the model can already sample [21], so we also

Preprint.

ask whether the gains come from problems the baseline solves in some of eight samples, or from problems it never solves.

So far, we have reproduced the baseline with the original compiler toolchain, which regenerates all 328 official HumanEval-C assembly files exactly. Two findings affect any comparison with GG. Library versions alone change the HumanEval-C result of the released model by five problems, or 3 percentage points. The authors' inference script also samples during beam search, while the paper states that sampling was disabled. Section 5 reports first results of GRPO and of an SFT baseline trained on the same problems.

Our proposal planned to apply DAPO to vision-language-action (VLA) models [28]. We changed the task for three reasons. None of us had used robot simulators before, and each training sample for a VLA is a full simulated episode [15, 19, 29]. We have one GPU rather than the two we planned for, which would have allowed too few such episodes. Finally, a simulated task gives one success signal after many actions and depends on the simulator, while a test run gives an exact result for a single generation within seconds. Assembly translation keeps the question of the proposal, reinforcement learning from outcomes after supervised fine-tuning, in a setting where both samples and rewards are cheap.

2 Related Work

Reinforcement learning with verifiable rewards. Reinforcement learning with verifiable rewards (RLVR) uses a program to check the answer instead of a learned reward model [13]. GRPO suits this setting because it compares several attempts at the same problem and does not require a separate model to predict their results [23]. The approach has achieved strong results on verifiable tasks such as mathematics and coding [5], and GRPO with binary rewards has been shown to increase the success probability of the starting model [21].

Verifiable rewards for code. Code is a natural fit for RLVR because tests give an exact and inexpensive result. RLEF trains code models on the results of running their own code [8], and VeRPO uses partial test results as a denser reward [26]. Recent work applies the same idea to low-level code, where the output can be executed or formally verified. SuperCoder uses reinforcement learning to produce faster assembly programs, rewarding both correctness and speed [27]. LLM-VeriOpt trains a small model with GRPO, using a formal verifier as the reward, to optimize compiler code [7]. AutoDecompiler uses compilation and test feedback to train a model that converts binaries back into source code [16], and TRAVEL applies reinforcement learning with execution feedback to translate C code into Rust [20]. These results show that RLVR can improve code, assembly, and translation. To our knowledge, none applies it to CISC-to-RISC assembly translation.

Assembly translation. Guess & Sketch combines guesses from a language model with a solver that checks whether the translation is equivalent [14]. CRT is a lightweight language model that translates x86 assembly into ARM and RISC-V [9]. GG fine-tunes small pre-trained code models (DeepSeek-Coder 1.3B and Qwen2.5-Coder 0.5B and 1.5B) on about 1.3 million C programs from AnghaBench [4] and The Stack v2 [18], compiled to both instruction sets. It generates candidate translations and checks them with software tests. It is evaluated on HumanEval, a set of programming problems whose C versions are translated and then executed, and on BringupBench, a set of larger programs for embedded systems [10]. In GG, the tests only filter the model's outputs. We use the same tests as a training signal.

3 Baseline

Our baseline is the released GG checkpoint for ARMv8 at O2, a Qwen2.5-Coder-1.5B-Instruct model [12] fine-tuned on the GG data [10]. We use it without further training. It is also the starting point of our method, so later improvements are measured against it.

Benchmarks and metric. HumanEval-C is the C version of HumanEval [3] released with LLM4Decompile [24]. It has 164 short functions, each with a test program of assertions. The model receives the x86 assembly of a function, and the assertions decide whether its ARM translation is

correct. BringUpBench [2] is a suite of complete C programs written to bring up new processors and compilers, which depend only on a small bundled library. Its programs are much longer than HumanEval functions, from 85 to 5,751 lines [10], and a program passes when its output matches the expected output. Following GG, we use the 65 programs that can be built. We report the percentage of problems solved with one prediction per problem, which is the same as pass@1 [3].

Evaluation harness. We implement our own harness so that evaluation and the training reward share the same code. The predicted assembly is the only definition of the function in the test program, which is assembled, linked, and run, so an empty or truncated prediction fails. The GG data targets macOS, so the harness runs on a Mac with Apple Silicon. Data regenerated with a recent Xcode differs from the official files, which were built with Xcode 14.3 and the macOS 13.3 SDK. We therefore built a macOS 13 virtual machine with that toolchain. Inside it, compiling the authors' C code reproduces all 328 official HumanEval-C assembly files exactly. As sanity checks, the reference ARM assembly passes all 164 problems, and an empty prediction fails all of them.

Results. Figure 1 gives the results. With beam search, as in the paper, the model solves 77 HumanEval-C problems with the library versions listed with the checkpoint and 72 with newer versions, compared with 74 reported by the authors. Decoding is identical in the two runs, and none of the problems that change reach the length limit, so the gap comes from the library versions alone. A 3-point difference could mask or mimic an improvement of the same size, so we evaluate the baseline and our models with fixed versions. On BringUp, the model solves 6 programs compared with the 10 reported, and the cause is still unknown. We also find that the authors' inference script enables sampling during beam search through the generation configuration of the model, although the paper states that sampling was disabled. Run as released, it solves 76 problems.

Table 1: Environments of our evaluations. Environment 1 has the library versions listed with the released model, and environment 2 is the one we use for training. †PyTorch 2.7 on BringUp, as required by the GPU used for that evaluation.

Environment	Transformers	PyTorch
1	4.50	2.6†
2	5.17	2.13

Table 2: Our evaluation runs of the released GG model (Figure 1).

Run	Environment	Decoding
A	1	Beam search
B	2	Beam search
C	1	Beam search with sampling
D	1	Greedy

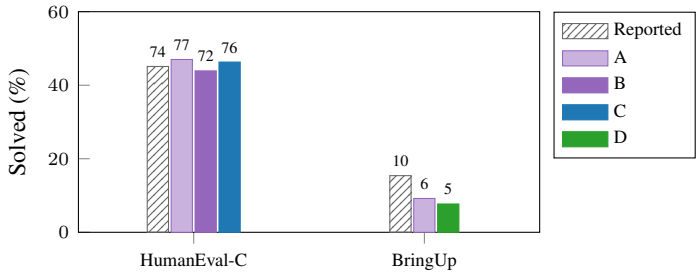


Figure 1: Problems solved by the released GG model at O2. The hatched bars are reported by Heakl et al. [10]. Runs A to D are our evaluations, listed in Table 2, with the number of problems solved above each bar. Beam search uses 8 beams and, as stated by Heakl et al. [10], no sampling. The released inference code samples during beam search on HumanEval-C and decodes greedily on BringUp. With a limit of 16,000 new tokens, run B also gives 72 problems.

4 Proposed Method

GRPO with unit test rewards. In SFT, the model learns to reproduce a reference translation token by token. GRPO instead learns from its own outputs. It never sees a reference translation, only whether the translations it generates pass the tests. For each training problem, the model samples $G = 8$ translations. Translation i gets reward $r_i = 1$ if it passes all tests and $r_i = 0$ otherwise, and its advantage is

$$\hat{A}_i = \frac{r_i - \mathrm{mean}(r_1, \ldots, r_G)}{\mathrm{std}(r_1, \ldots, r_G)}, \tag{1}$$

applied to every token of the translation [23]. The advantage measures how much better translation i is than the average translation of its group. It is positive for a passing translation when some others fail, and negative for a failing one when some others pass. The loss multiplies the log-probability of each token by the advantage of its translation, so an update makes translations with a positive advantage more likely and those with a negative advantage less likely. Passing translations become more likely relative to failing ones, and groups in which all translations pass or all fail do not contribute. In such a group every r_i equals the mean, so every $\hat{A}_i$ is zero (the implementation adds a small constant to the standard deviation, so the division stays defined), and the group does not change the weights. We update all weights of the model. At 1.5 billion parameters, the model, its gradients, and the optimizer states fit on our 96 GB GPU with 32-bit master weights and 16-bit computation, so neither low-rank adapters nor quantization is needed. As in the original GRPO [23], we add the KL penalty to the loss. The KL divergence measures how much two probability distributions differ. The KL penalty grows as the token probabilities of the trained model move away from those of the reference model, a frozen copy of the starting model, so it keeps the two close. The reference model also fits in GPU memory. We use the GRPO trainer of TRL [25] and write the reward function, data preparation, and evaluation ourselves.

Training data. MBPP [1] contains short programming problems with assertion tests. We use a C version of it from LLM4Decompile [24], compiled in the same way as the GG data. Of its 973 problems that compile, we keep the 952 whose reference ARM assembly passes its own tests, and split them by distinct x86 program into 858 training and 94 validation problems. HumanEval-C is used only for evaluation. Heakl et al. [10] do not evaluate on MBPP, so no published GG results exist for it.

Settings. Each step uses 16 problems with eight translations each, and training runs for 106 steps, two epochs. Translations are limited to 4,096 tokens, which covers the reference of all but 11 training problems. Truncated translations stay in the loss with a reward of 0. The loss of a translation is the average over its tokens, and the loss of a step averages over the translations, so a token of a short translation counts more than a token of a long one. The KL penalty has weight $\beta = 0.04$, and the learning rate is 10^{-6} and decays linearly to zero. We use gradient accumulation because only 4 translations fit in GPU memory at a time, so each step adds up the gradients of 32 micro-batches of 4 translations. Unless stated otherwise, GRPO refers to this setup. We evaluate the checkpoint after the last step and do not use the validation set to choose a checkpoint. For the SFT baseline, we fine-tune the released model on the reference translations of the same training problems, with the settings of the released GG run (learning rate 2×10^{-5}, cosine decay, two epochs) and 16 problems per step as in GRPO. These settings are not tuned.

Progress and risks. Training runs on a Linux server with one NVIDIA RTX PRO 6000 Blackwell GPU (96 GB), which sends the generated translations through a network tunnel to the Mac and receives the rewards. Two risks remain. Reward computation on a single Mac limits the training budget. Updating all weights may let the model drift or lose the diversity of its samples, so we will track the entropy of the samples during training.

5 Preliminary Results

5.1 Differences from GG

Our setup differs from that of Heakl et al. [10] in the training data, the validation set, the benchmark subsets and the evaluation. The validation set and the two benchmark subsets are ours, so their numbers have no counterpart in that work.

GG is trained with SFT on about 1.3 million programs from AnghaBench [4] and The Stack v2 [18]. We start from the released model and train it further, with GRPO or with SFT for comparison, on 858 MBPP-C problems, which were not used to train or evaluate GG. Of the 865 MBPP-C problems long enough to check, only 25 appear in the GG training data as C code and 76 as assembly, so most of these problems are new to the model.

We hold out 94 MBPP-C problems as a validation set to monitor training (Appendix B). HumanEval-C and BringUp are used only for the final evaluation. We do not use them to choose settings or checkpoints, and we evaluate them only on the final checkpoint of each training run.

We also check for data contamination, that is, benchmark problems that also appear in training data. We find that 13 MBPP-C problems are the same as 12 HumanEval-C problems, and that 12 of the 65 BringUp programs appear in the GG training data. We therefore also check HumanEval-C without these 12 problems (Section 5). None of the BringUp programs solved by the base model, GRPO or SFT is among these 12 programs, so they do not affect the BringUp results.

For decoding, we use beam search with 8 beams and no sampling, as Heakl et al. [10] state, although the released inference script samples during beam search (Section 3). Library versions change the result on HumanEval-C by up to five problems, so we evaluate all models with the same fixed versions. We use environment 2 of Table 1 for HumanEval-C and the validation set, and environment 1 for BringUp. We also use our own test harness, with the authors' tests and the original compiler toolchain (Section 3).

5.2 Main Results

Figure 2 shows the two benchmarks used by Heakl et al. [10]. GRPO raises the percentage of HumanEval-C problems solved from 43.9% to 51.2%. Without the problems that also appear in MBPP-C, it rises from 42.8% to 50.0%, so these problems do not explain the gain. Each method is trained with a single seed, so we cannot yet estimate the variance across seeds.

GRPO solves 14 problems that the base model failed and fails 2 that it solved. The base model already solves 12 of these 14 problems in at least one of eight samples at temperature 1, and with more samples it might solve some of the others too.

Because MBPP-C is new to the model, part of the gain could come from seeing new problems rather than from the reward. The SFT baseline, trained on the same problems, controls for this, and it does not improve HumanEval-C. It solves 10 problems that the base model failed and fails 17 that it solved. Exposure to new problems therefore does not explain the GRPO gain, at least with this SFT setup. Before SFT, the model already predicts 99.0% of the tokens of the reference translations, so it has little to learn from them.

No model changes BringUp by more than two programs. The base model solves 6 of 65, GRPO solves 5, and SFT solves 4. Differences of one or two programs are too small to be conclusive.

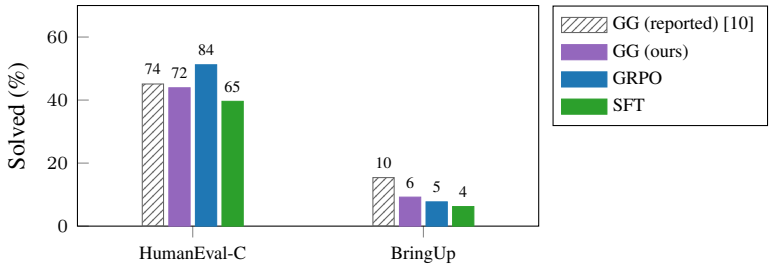


Figure 2: Percentage of problems solved with beam search (8 beams) on the two benchmarks used by Heakl et al. [10], at O2, with the number solved above each bar. The hatched bar is the result reported by Heakl et al. [10]. All other bars are our evaluations, with environment 2 of Table 1 for HumanEval-C and environment 1 for BringUp.

6 Remaining Work and Task Distribution

Goal	Lead
Train the GRPO variants, and train each method with three random seeds	Enzo Shiraishi
Evaluate other RLVR techniques	Enzo Shiraishi
Evaluate the baselines and the trained models on HumanEval-C and BringUp	Zayed Al Tamimi
Add training problems with official tests	Sergei Pasynkov
Run the ablation with a graded reward	Sergei Pasynkov
Run the parameter study on the group size or the learning rate	Zayed Al Tamimi
Analyze results and failures, and write the final report	All

The final report will compare our method with at least three baselines, namely the released GG model with beam search, the same model with best-of-eight sampling filtered by the tests, and supervised fine-tuning on the reference translations of the same MBPP problems. The ablation will replace the binary reward with a graded one that gives partial credit for assembly that assembles and links, and the parameter study will vary the group size or the learning rate.

Implementations of GRPO differ in details such as how the loss is averaged, so we will also compare with Dr. GRPO [17], which divides the loss by a constant instead of by the length of each translation and does not divide the advantage by the standard deviation. We will also train the default setup of TRL [25], which averages the loss over all tokens of a step instead of per translation, leaves truncated translations out of the loss, and has no KL penalty. We will train every method with three random seeds and tune the hyperparameters of SFT.

In 59.1% of the groups during GRPO training, all eight translations pass or all fail, so they give no gradient (Appendix A), and MBPP-C has only 858 training problems. We will add problems only from sources with official judge tests, which gives 890 Codeforces problems [11] and up to 1,832 AOJ problems [22], about 2.5 times the size of MBPP-C. These problems are judged on their printed output, so the reward needs an output-comparison mode.

We also want to try other RLVR methods for the groups that give no gradient, such as the dynamic sampling of DAPO, which only keeps groups with both passing and failing translations [28], or larger groups. We also plan to compare with rejection-sampling fine-tuning, which trains the model on its own passing translations [30, 6].

We will sample the base model more than eight times on the problems that GRPO gains, look for the reason why we solve 6 BringUp programs compared with the 10 reported, and study why GRPO solves more problems with beam search on HumanEval-C but not on the validation set.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language

models. *arXiv preprint arXiv:2108.07732*, 2021. URL https://arxiv.org/abs/2108.07732.

[2] Todd Austin. Bringup-Bench: A benchmark suite for bringing up new processors and compilers. GitHub repository, 2024. URL https://github.com/toddmaustin/bringup-bench.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL https://arxiv.org/abs/2107.03374.

[4] Anderson Faustino da Silva, Bruno Conde Kind, José Wesley de Souza Magalhães, Jerônimo Nunes Rocha, Breno Campos Ferreira Guimarães, and Fernando Magno Quintão Pereira. AnghaBench: A suite with one million compilable C benchmarks for code-size reduction. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021.

[5] DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *Nature*, 645:633–638, 2025. URL https://arxiv.org/abs/2501.12948. arXiv:2501.12948.

[6] Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. RAFT: Reward ranked finetuning for generative foundation model alignment. *Transactions on Machine Learning Research*, 2023. URL https://arxiv.org/abs/2304.06767.

[7] X. Fang, J. Kang, R. C. Rocha, S. Ainsworth, and L. Mukhanov. LLM-VeriOpt: Verification-Guided Reinforcement Learning for LLM-Based Compiler Optimization. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2026. URL https://2026.cgo.org/details/cgo-2026-papers/37/LLM-VeriOpt-Verification-Guided-Reinforcement-Learning-for-LLM-Based-Compiler-Optimi.

[8] J. Gehring, K. Zheng, J. Copet, V. Mella, Q. Carbonneaux, T. Cohen, and G. Synnaeve. RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning. *arXiv preprint arXiv:2410.02089*, 2024. URL https://arxiv.org/abs/2410.02089.

[9] A. Heakl, C. Abi, R. Hossam, and A. Mahmoud. From CISC to RISC: Language-Model Guided Assembly Transpilation. *arXiv preprint arXiv:2411.16341*, 2024. URL https://arxiv.org/abs/2411.16341.

[10] A. Heakl, S. Hashmi, C. Abi, C. Lee, and A. Mahmoud. Guaranteed Guess: A Language Modeling Approach for CISC-to-RISC Transpilation with Testing Guarantees. *arXiv preprint arXiv:2506.14606*, 2025. URL https://arxiv.org/abs/2506.14606.

[11] Hugging Face Open-R1. Codeforces: problems, official tests and submissions. Hugging Face datasets open-r1/codeforces and open-r1/codeforces-submissions, 2025. URL https://huggingface.co/datasets/open-r1/codeforces.

[12] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. URL https://arxiv.org/abs/2409.12186.

[13] N. Lambert et al. Tulu 3: Pushing Frontiers in Open Language Model Post-Training. *arXiv preprint arXiv:2411.15124*, 2024. URL https://arxiv.org/abs/2411.15124.

[14] C. Lee, A. Mahmoud, M. Kurek, S. Campanoni, D. Brooks, S. Chong, G.-Y. Wei, and A. M. Rush. Guess & Sketch: Language Model Guided Transpilation. *arXiv preprint arXiv:2309.14396*, 2023. URL https://arxiv.org/abs/2309.14396.

[15] H. Li et al. SimpleVLA-RL: Scaling VLA Training via Reinforcement Learning. *arXiv preprint arXiv:2509.09674*, 2025. URL https://arxiv.org/abs/2509.09674.

[16] P. Liu et al. Binary Decompilation LLM with Feedback-Driven Multi-Turn Refinement. *arXiv preprint arXiv:2606.16162*, 2026. URL https://arxiv.org/abs/2606.16162.

[17] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025. URL https://arxiv.org/abs/2503.20783.

[18] Anton Lozhkov, Raymond Li, Loubna Ben Allal, et al. StarCoder 2 and The Stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024.

[19] G. Lu et al. VLA-RL: Towards Masterful and General Robotic Manipulation with Scalable Reinforcement Learning. *arXiv preprint arXiv:2505.18719*, 2025. URL https://arxiv.org/abs/2505.18719.

[20] F. Luo, J. Liu, C. Gao, J. Feng, and K. Liu. Towards Reliable C-to-Rust Translation with Rule-Guided Reasoning and Reinforcement Learning. *arXiv preprint arXiv:2607.19966*, 2026. URL https://arxiv.org/abs/2607.19966.

[21] Y. Mroueh. Reinforcement Learning with Verifiable Rewards: GRPO's Effective Loss, Dynamics, and Success Amplification. *arXiv preprint arXiv:2503.06639*, 2025. URL https://arxiv.org/abs/2503.06639.

[22] Ruchir Puri, David S. Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, Veronika Thost, Luca Buratti, Saurabh Pujar, Shyam Ramji, Ulrich Finkler, Susan Malaika, and Frederick Reiss. CodeNet: A large-scale AI for code dataset for learning a diversity of coding tasks. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021. URL https://arxiv.org/abs/2105.12655.

[23] Z. Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.

[24] Hanzhuo Tan, Qi Luo, Jing Li, and Yuqun Zhang. LLM4Decompile: Decompiling binary code with large language models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. URL https://arxiv.org/abs/2403.05286.

[25] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformer reinforcement learning. GitHub repository, 2020. URL https://github.com/huggingface/trl.

[26] L. Wang et al. Beyond Binary: Turning Partial Success into Dense Verifiable Rewards for Reinforcement Learning in Code Generation. *arXiv preprint arXiv:2601.03525*, 2026. URL https://arxiv.org/abs/2601.03525.

[27] A. Wei, T. Suresh, H. Tan, Y. Xu, G. Singh, K. Wang, and A. Aiken. SuperCoder: Assembly Program Superoptimization with Large Language Models. *arXiv preprint arXiv:2505.11480*, 2025. URL https://arxiv.org/abs/2505.11480.

[28] Q. Yu et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *arXiv preprint arXiv:2503.14476*, 2025. URL https://arxiv.org/abs/2503.14476.

[29] H. Zang et al. RLinf-VLA: A Unified and Efficient Framework for Reinforcement Learning of Vision-Language-Action Models. *arXiv preprint arXiv:2510.06710*, 2025. URL https://arxiv.org/abs/2510.06710.

[30] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2203.14465.

A Training

At every step, the model samples 8 translations for each of 16 training problems, and the reward server tests them. Figure 3 shows two quantities, averaged over 10 steps. The pass rate is the fraction of the 128 translations of a step that pass all tests. It averages 45.5% in the first epoch and 50.4% in the second, so the model improves on its training problems. A group gives no gradient when all eight translations pass or all fail. In that case every reward equals the mean of the group, every advantage in Equation (1) is zero, and the group does not change the weights. This happens in 59.1% of the groups, so more than half of the problems of a step do not contribute to it.

For SFT, Figure 4 shows the loss on the reference translations and the share of their tokens that the model predicts correctly. The token accuracy is already 99.0% at the first step and ends at 99.6% on average over the last 10 steps, so the released model already reproduces the reference translations almost exactly. The loss rises during the first epoch, while the learning rate is highest, and falls during the second, from 0.032 at the first step to 0.019 on average over the last 10 steps. The drop of the validation pass rate after the first epoch (Appendix B) coincides with this rise.

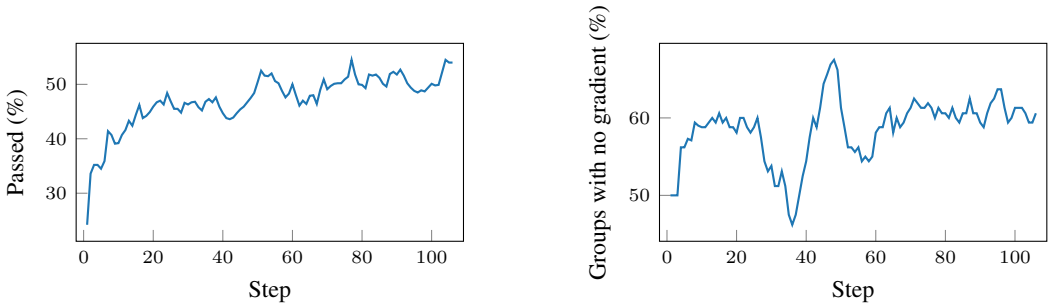


Figure 3: Training curves, averaged over 10 steps. Left: pass rate of the 128 translations of a step. Right: groups in which all eight translations pass or all fail, which give no gradient.

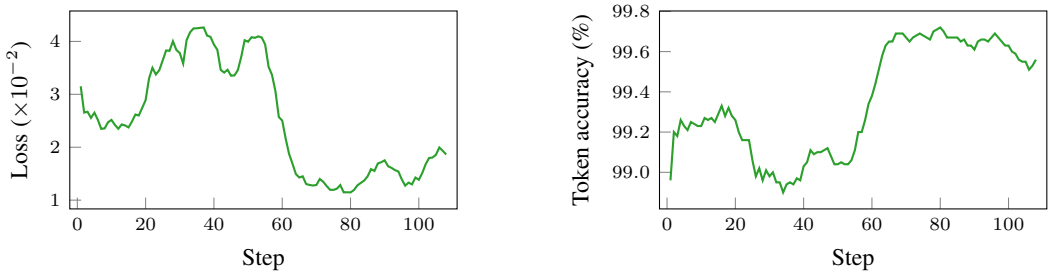


Figure 4: Training of SFT, averaged over 10 steps. Left: loss on the reference translations. Right: share of the reference tokens predicted correctly.

B Validation Set

The validation set has 94 MBPP-C problems held out from training, from the same source as the training data. We use it only to monitor training, not to choose a checkpoint. We report the percentage of problems solved with beam search, as for the benchmarks, and with sampling at temperature 1, averaged over 8 samples per problem (Figure 5). With beam search, the base model solves 59.6% of the problems and GRPO 56.4%, and SFT 58.5%, so the result with beam search is essentially unchanged, unlike on HumanEval-C. With sampling, GRPO goes from 45.1% to 47.6%. SFT drops to 33.9% after its first epoch and ends at 43.5%. Of its 752 samples, 126 fail to assemble, compared with 80 for the base model and 92 after GRPO.

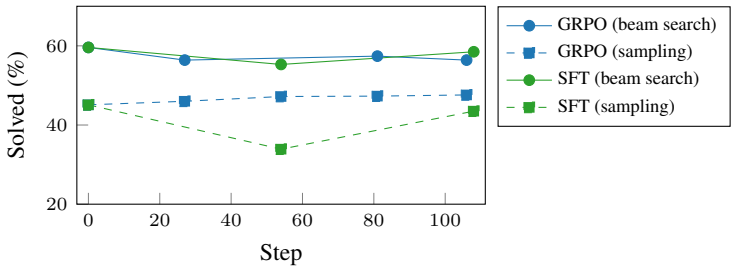


Figure 5: Percentage of the 94 validation problems solved at each checkpoint, with beam search (solid) and with sampling at temperature 1 (dashed). Step 0 is the base model.